青岛农业大学海都学院

本科生毕业论文（设计）

题　　目：基于SpringBoot的汽车车险销售系统的设计与实现

姓　　名：张浩

系　　别：信息工程学院

专　　业：计算机科学与技术

班　　级：2022级1班

学　　号：20221836025

指导教师：张琛

2026年5月12日

学士学位论文（设计）原创性声明

本人郑重声明：所呈交的学位论文（设计），是本人在导师的指导下进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在论文中做了明确的说明并表示了谢意。本声明的法律后果由本人承担。

学位论文（设计）作者签名：张浩

2026 年 2 月 2 日

关于论文（设计）使用授权的说明

本人完全了解青岛农业大学海都学院有关保留、使用学士学位论文的规定，即：学校有权保留、送交论文的复印件，允许论文被查阅，学校可以公布论文的全部或部分内容，可以采用影印或其他复制手段保存论文。

指导教师签名：张琛　　论文作者签名：张浩

2026 年 2 月 2 日　　年　月　日

目　　录

摘要：..... 1
Abstract.....2
1 概述..... 3
1.1 课题研究背景和意义..... 3
1.2 国内外研究现状..... 3
1.3 课题主要研究内容..... 4
1.4 论文组织结构..... 4
2 需求分析..... 4
2.1 可行性分析..... 4
2.1.1 经济可行性分析..... 4
2.1.2 技术可行性分析..... 4
2.2 功能性需求分析..... 4
2.2.1 系统功能概述..... 4
2.2.2 系统功能需求分析..... 5
2.3 非功能性需求分析..... 7
2.3.1 系统性能需求分析..... 7
2.3.2 开发环境..... 8
3 概要设计..... 8
3. 1 系统架构设计..... 8
3. 2 系统功能模块设计..... 10
3.2.1 业务员功能模块..... 10
3.2.2 用户功能模块..... 11
3.2.3 管理员功能模块..... 12
3. 3 数据库设计..... 13
3.3.1 概念结构设计..... 13
3.3.2 逻辑结构设计..... 18
4 详细设计与实现..... 24
4.1 系统功能设计..... 24
4.1.1 业务员端功能详细设计..... 25
4.1.2 用户端功能详细设计..... 26

4.1.3 管理员端功能详细设计 29
4..2 系统功能实现 32
4.2.1 用户功能的设计与实现 32
4.2.2 业务人员的功能设计与实现 39
4.2.3 管理员功能的设计与实现 42
5 系统测试 45
5.1 订单创建功能测试 45
5.2 理赔申请功能测试 45
5.3 救援申请功能测试 46
5.4 智能算价功能测试 47
6 总结 47
参考文献 49
致谢 50

基于微信小程序的车辆保险服务平台的设计与实现

计算机科学与技术 张浩

摘要：随着机动车保有量持续增长，车险业务的线上化、便捷化与智能化需求不断提升。针对传统车险业务在投保、支付、出单、理赔与救援环节中存在流程割裂、人工参与度高、服务反馈慢等问题，本文设计并实现了一套车险销售与服务一体化系统。系统采用“后端服务+管理后台+微信小程序”三端协同架构：后端基于Spring Boot与MyBatis-Plus构建RESTful接口，结合Spring Security与JWT完成身份认证；管理后台基于Vue3与Element Plus实现用户、产品、车辆、订单和理赔管理；小程序端为普通用户提供产品浏览、智能算价、在线投保、理赔申请、道路救援、违章查询、家庭成员管理与电子证件钱包等功能。数据库采用MySQL，围绕用户、车辆、保险产品、订单、理赔、救援、证件和家庭成员建立核心数据模型。系统实现了订单支付后自动生成电子保单并写入钱包的联动机制，形成较完整的业务闭环。测试结果表明，系统核心功能运行稳定，能够满足车险销售与服务场景的基本业务需求。

关键词：前后端分离；车险系统；Spring Boot；

Construction and Implementation of an Integrated Car Insurance Sales and Service System Based on Spring Boot and Vue3

Computer Science and Technology　Zhang Hao

Abstract: With the rapid growth of vehicle ownership, digital car insurance services are increasingly required. To solve the problems of fragmented workflow, high manual effort, and slow feedback in traditional insurance services, this paper designs and implements an integrated car insurance sales and service system. The system adopts a three-end architecture including backend services, an admin web portal, and a WeChat mini program. The backend is built with Spring Boot and MyBatis-Plus, and authentication is implemented using Spring Security and JWT. The admin portal is developed with Vue3 and Element Plus, supporting user, product, vehicle, order, and claim management. The mini program provides product browsing, premium quotation, order creation, claim submission, roadside rescue, violation inquiry, family member management, and digital document wallet. MySQL is used to model core entities and relationships. The system supports automatic electronic policy generation and wallet linkage after payment, forming an end-to-end business process. Test results show that the core functions run stably and meet practical business requirements.

Key words: Front and back end separation; Car Insurance System; SpringBoot

1 概述

1.1 课题研究背景和意义

随着机动车保有量持续增长，车险业务正由传统线下办理向线上化、移动化和智能化方向演进，用户对“随时可办、流程透明、反馈及时”的服务需求不断提高[3][13]。现有部分系统在投保、核保、支付、理赔、救援等环节仍存在模块割裂、重复录入、状态不同步等问题，导致业务处理链路长、人工成本高、用户体验不稳定[7][10]。

基于此，本文围绕真实项目 car-insurance-system，构建“管理后台 + 小程序端 + 后端服务”的三端协同平台，打通投保前咨询、在线投保支付、保后服务等关键流程，提升业务办理效率与服务质量[3][9]。在技术路线上，系统采用 Spring Boot + Vue + 微信小程序的前后端分离架构，结合统一接口规范与分层设计，保障系统可扩展、可维护[4]。同时通过数据采集与处理能力，为后续接入外部违章、OCR 识别与风控分析预留能力[8][12]。

1.2 国内外研究现状

1.2.1 国外研究现状

国外在软件系统工程化与智能化方面积累较为成熟，研究重点从单点功能实现逐步转向“架构可扩展 + 自动化质量保障 + 持续迭代优化”[15]。在系统质量保障方面，自动化测试与智能测试策略持续发展，为复杂业务系统的稳定交付提供了方法参考[14]。这些研究对车险平台建设的启示在于：需要在业务功能之外，同步重视架构治理、测试体系和长期演进能力，以支撑多角色、多流程、高频交互的系统场景。

1.2.2 国内研究现状

国内围绕微信小程序、Spring Boot、SSM 等技术栈开展了大量管理类系统研究，已形成较成熟的三端协同开发经验[3][7]。相关研究在功能模块划分、接口设计、权限控制、数据库建模和业务流程闭环方面给出了可复用实践，为本课题中“投保—支付—理赔—救援”一体化流程设计提供了参考[9][11]。同时，数据采集与处理相关研究也为外部数据接入、数据分析与服务优化提供了支撑思路[8][12]。总体来看，国内研究具备良好的工程落地基础，但在跨系统数据融合、流程自动化深度和质量保障体系方面仍有提升空间，这也是本课题的重点改进方向。

1.3 课题主要研究内容

（1）分析管理员、业务员、普通用户三类角色需求；

（2）完成后端、管理端、小程序端三端协同架构设计；

（3）实现认证鉴权、产品管理、车辆管理、订单管理、理赔管理、智能算价、救援服务、违章查询、家庭成员、电子钱包等核心模块；

（4）构建数据库模型并完成关键实体映射；

（5）完成系统功能测试与算价单元测试，评估系统可用性。

1.4 论文组织结构

本文共分为六章。第一章介绍研究背景与意义；第二章进行需求分析；第三章完成概要设计；第四章阐述系统详细设计与实现；第五章给出系统测试过程与结果；第六章总结全文并提出后续优化方向。

2 需求分析

2.1 可行性分析

2.1.1 经济可行性分析

系统采用开源技术栈与通用开发工具，部署门槛低、维护成本可控，能够在本科毕业设计周期内完成开发与测试工作，具有较好的经济可行性。

2.1.2 技术可行性分析

后端采用 Spring Boot + MyBatis-Plus，前端采用 Vue3 + Element Plus，小程序采用微信原生框架，数据库使用 MySQL。该技术方案成熟稳定，社区文档完善，能够支持本课题功能实现。

2.2 功能性需求分析

2.2.1 系统功能概述

本车险系统采用多端协同架构，包含管理后台（Web 端）与用户小程序端两大核心端，并通过统一的后端服务与数据库实现数据同步与业务协同。系统核心目标是打通“用户投保—订单支付—保单管理—理赔服务—救援/辅助服务”的完整业务闭环，满足普通用户便捷投保、业务人员高效管理、管理员规范监管的需求。

其中，管理后台面向管理员与业务员，重点覆盖用户管理、产品管理、车辆

管理、订单管理、理赔管理、救援记录与系统配置等核心运营功能，确保业务流程规范、数据可追溯、风险可控；用户小程序端面向普通车主用户，提供产品浏览、在线算价、投保下单、支付、理赔申请、救援服务、违章查询、证件管理等功能，提升投保效率与服务体验。

两端通过后端接口实现用户信息、车辆信息、投保订单、理赔状态与救援记录的实时一致，保障系统稳定运行与业务连续性。

2.2.2 系统功能需求分析

系统功能需求围绕“管理员端、业务员端、用户小程序端”三类核心角色展开，结合各角色的业务定位，明确模块功能点、交互逻辑与实现要求，

具体如下：

（1）管理员端功能需求分析

管理员端目标是实现平台级治理与全局监管，保障业务规范运行与数据可溯。

管理员用例图如图 2-1 所示。

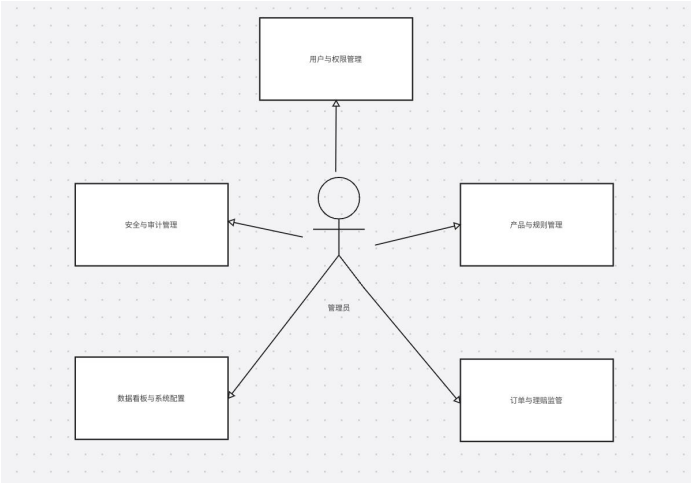

图 2-1 管理员用例图

功能需求包括：

a. 用户与权限管理：支持用户信息查询、账号启停、角色分配、权限控制与关键操作审计。

b. 产品与规则管理：支持保险产品新增/编辑/上下架，维护保障范围、费率规则、适用车型与投保条件，支持版本管理与历史追溯。

c. 订单与理赔监管：支持订单全量查询、状态监控、异常订单处理；支持理赔复核、审批跟踪与归档管理。

d. 数据看板与系统配置：提供投保量、保费收入、理赔率等统计看板；支持费率参数、通知模板、系统基础参数配置。

e. 安全与审计管理：支持操作日志查询、异常行为告警与风控策略执行，确保平台安全可控。

（2）业务员端功能需求分析

业务员端目标是提升一线业务处理效率，打通“客户—订单—理赔—救援”协同流程。业务员用例图如图 2-3 所示。

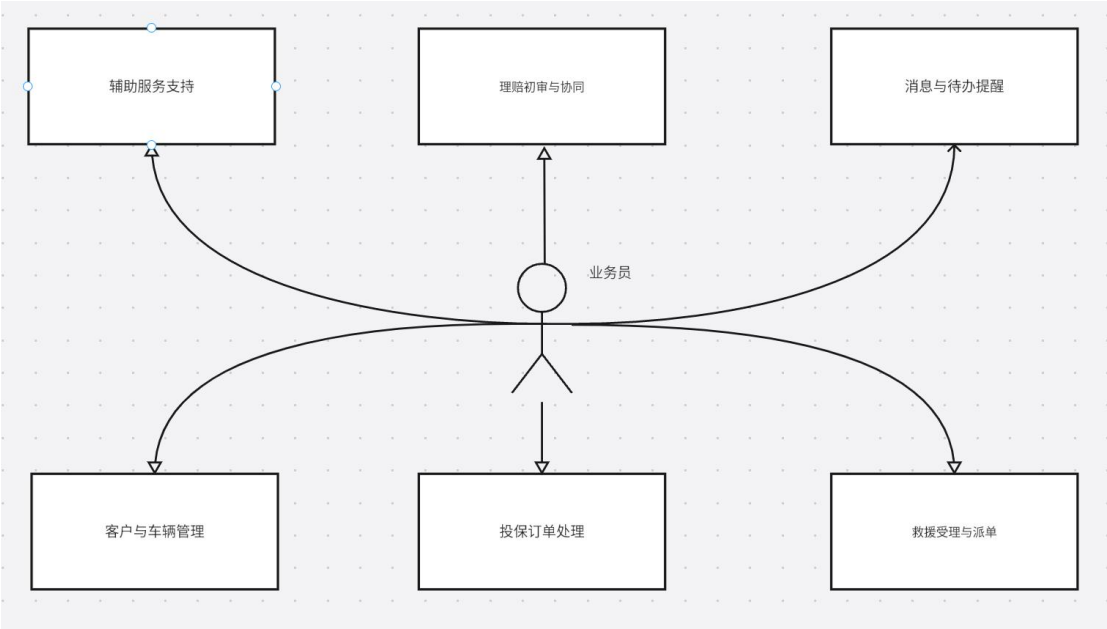

图 2-2 业务员用例图

功能需求包括：

a. 客户与车辆管理：支持按手机号/用户 ID 检索客户信息，完成车辆信息核验、证件校对与审核处理。

b. 投保订单处理：支持订单详情核查、审核通过/退回、状态流转跟踪与异常情况反馈。

c. 理赔初审与协同：支持理赔资料初审、意见提交、节点跟进与处理状态同步。

d. 救援受理与派单：支持救援请求查看、位置确认、派单处理与状态更新。

e. 辅助服务支持：支持违章查询协助、证件识别纠错、资料补录与业务备注管理。

f. 消息与待办提醒：支持待审核订单、待处理理赔、救援工单等任务提醒，提升处理及时性。

（3）用户小程序端功能需求分析

用户端面向普通车主，核心目标是提供便捷投保、快速理赔与服务闭环体验。

车主用户用例图如图 2-3 所示。

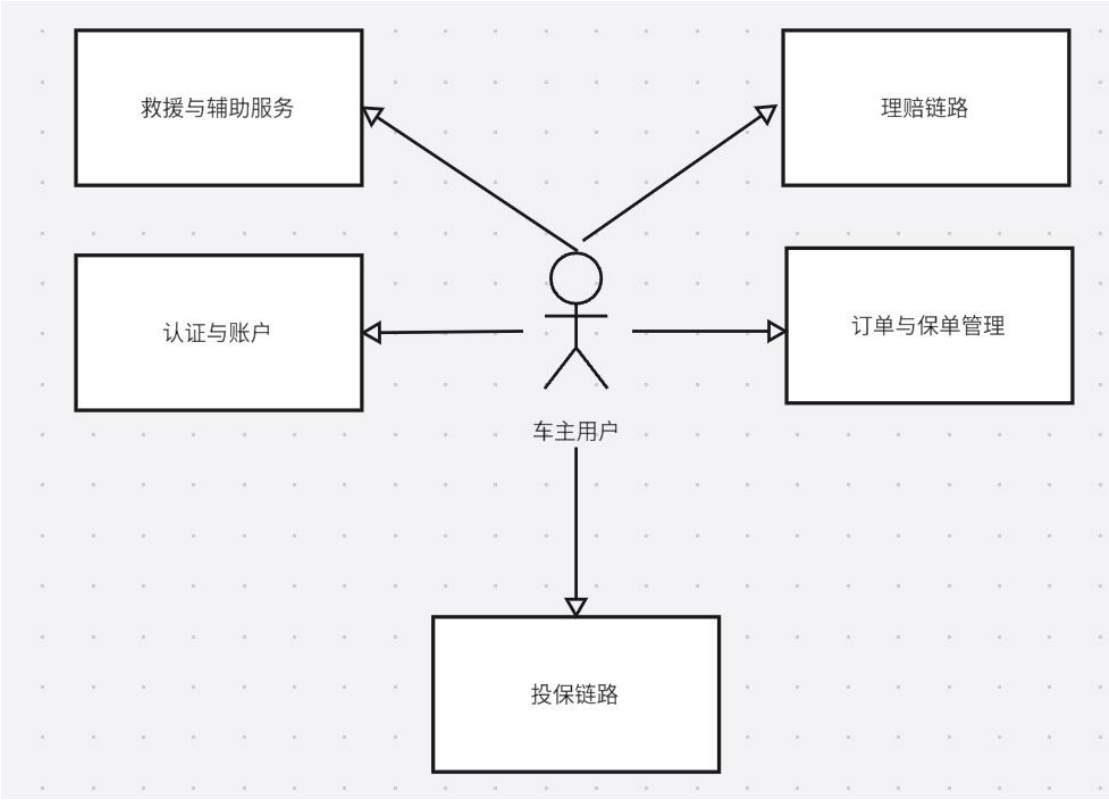

图 2-3 车主用户用例图

功能需求包括：

a. 认证与账户：支持手机号注册登录、验证码校验、密码找回与个人信息维护。

b. 投保链路：支持产品浏览筛选、车辆选择/录入、在线算价、投保下单与在线支付。

c. 订单与保单管理：支持订单状态查询、保单查看、历史订单检索与电子保单展示。

d. 理赔链路：支持理赔申请提交、材料上传、进度查询与结果通知。

e. 救援与辅助服务：支持救援类型选择、位置提交、历史记录查询；支持违章查询、电子证件上传与展示。

以上功能共同构成系统业务闭环，确保投保、理赔与救援服务流程可控、可追踪、可持续优化。

2.3 非功能性需求分析

2.3.1 系统性能需求分析

系统需保证高频查询接口稳定响应，常规操作应在可接受时间内返回结果；

重点保障订单、理赔、产品查询等核心业务接口的可用性和稳定性。

2.3.2 开发环境

后端开发环境采用 JDK 11，使用 IntelliJ IDEA 作为开发工具，通过 Maven 3.8.x 管理依赖。在 pom.xml 中引入 spring-boot-starter-web、spring-boot-starter-securit、spring-boot-starter-validation、mybatis-plus-boot-starter、mysql-connector-java、jjwt 等依赖，实现 Web 服务、权限认证、数据访问与 JWT 登录。application.yml 中配置服务端口 8081、MySQL 数据源 car_insurance、MyBatis-Plus 映射规则、JWT 密钥与过期时间、日志输出及 SQL 初始化脚本（update_schema_v2.sql）。

前端 Web 端开发环境采用 Node.js 18+，使用 VS Code 作为开发工具，基于 Vue3 + Vite 开发，依赖 vue-router 实现路由管理、pinia 实现状态管理、axios 实现 HTTP 请求、element-plus 实现 UI 组件、echarts 实现数据可视化。

移动端采用微信小程序开发工具，项目位于 miniprogram 目录，通过 app.js 配置后端基址 baseUrl，所有请求统一由 utils/request.js 封装并自动携带 token。

数据库采用 MySQL 8.0，通过 Spring SQL 初始化脚本自动创建表结构与基础数据。

3 概要设计

3.1 系统架构设计

系统采用前后端分离架构，后端统一提供 REST API，管理端与小程序端按角色访问接口。后端分层为 Controller、Service、Mapper、Entity，并通过统一响应模型返回结果，系统采用 JWT 实现无状态认证，提高接口扩展能力。

本车险系统采用前后端分离架构，兼顾多端协同需求，后端基于 Spring Boot 提供接口服务，前端基于 Vue3 与微信小程序分别开发管理端与用户端，各端通过 HTTP 请求与后端接口交互，实现数据同步与功能协同。系统整体分为表现层、业务逻辑层、数据访问层、数据存储层四层架构，架构设计简洁清晰，各层职责明确、低耦合，便于后续系统开发、测试与维护，系统架构图如 3-1 下：

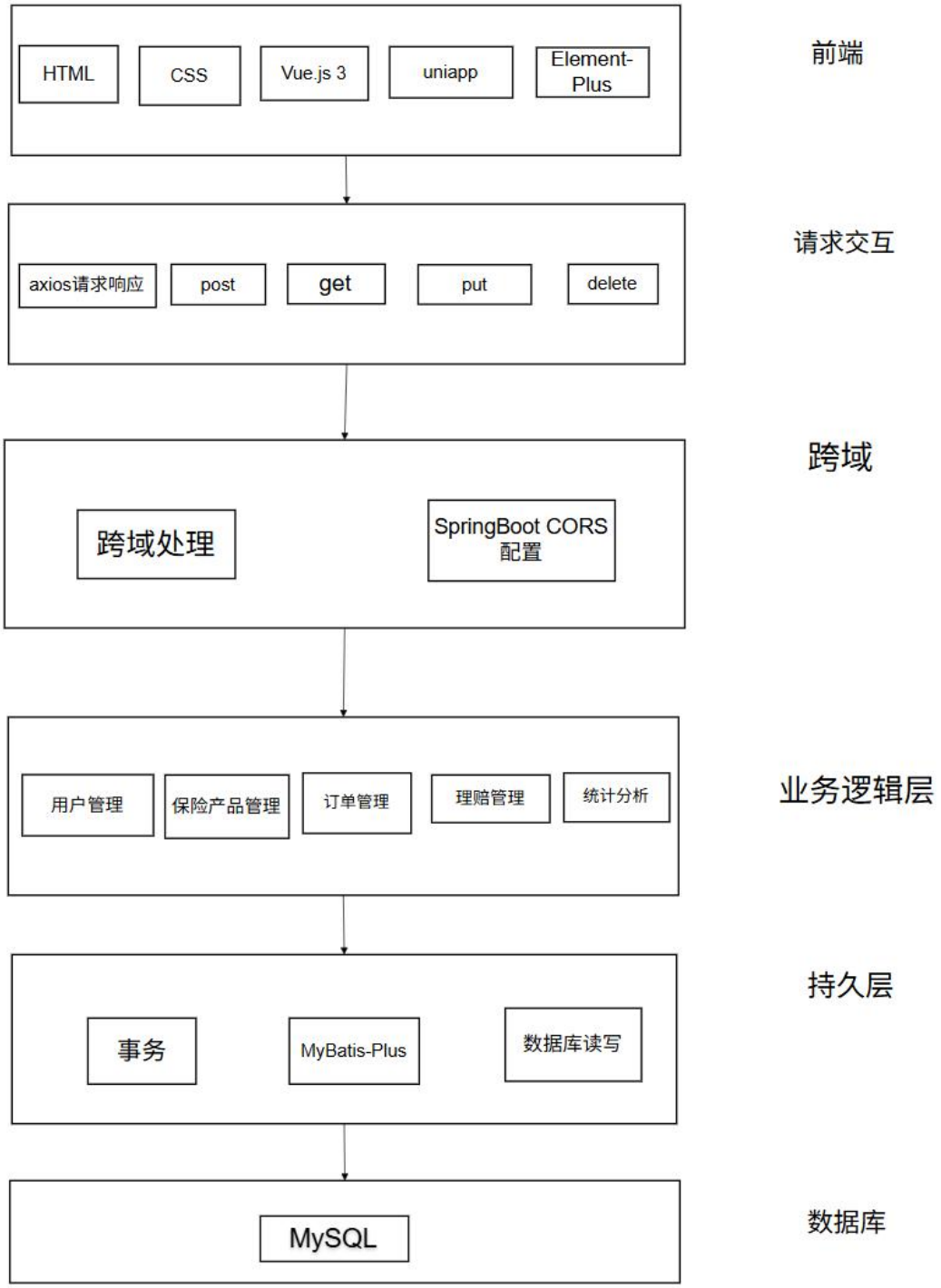

图 3-1 技术架构图

各层具体职责如下：

(1) 表现层：作为系统与用户的交互入口，负责接收用户操作请求、展示系统数据与操作结果，适配不同用户端的使用需求。其中，管理端采用 Vue3 开发网页端，适配电脑端操作；用户端采用微信小程序开发，适配手机端使用，实现多端协同。表现层不处理复杂业务逻辑，仅负责请求转发与结果渲染。

(2) 业务逻辑层：系统核心层，负责处理所有业务逻辑，衔接表现层与数据访问层。根据需求分析梳理功能模块，拆分出认证、用户、产品、车辆、订单、理赔、救援、违章、家庭成员、电子钱包等核心模块，各模块独立实现对应业务逻辑（如投保下单、理赔审核、救援申请），同时实现模块间的协同交互（如订单创建后生成保单、理赔提交后同步审核流程），确保业务流程合规、高效运行。

(3) 数据访问层：负责实现业务逻辑层与数据存储层的数据交互，封装数据访问接口，提供数据查询、新增、修改、删除等基础操作。采用 MyBatis-Plus 统一接口封装规范，实现异常处理与统一返回格式，降低与业务逻辑层、数据存储层的耦合度。

(4) 数据存储层：负责系统所有数据的持久化存储，保障数据安全、完整、可高效访问。核心采用 MySQL 关系型数据库，存储用户信息、车辆信息、产品信息、订单信息、理赔信息等核心业务数据；可引入缓存机制存储热点数据与会话信息，提高系统响应速度、降低数据库压力。

3.2 系统功能模块设计

基于系统架构设计和前期功能性需求分析，结合业务员、用户端、管理端三大核心端的用户定位，对系统功能模块进行细化设计，明确各模块的实现逻辑、交互流程，确保模块划分合理、功能贴合需求。系统功能模块采用模块化设计思想，各端模块独立开发、协同工作，便于后续功能扩展与维护，总体分为三大核心模块组，对应三大用户端，功能模块图如图 3-2 所示。

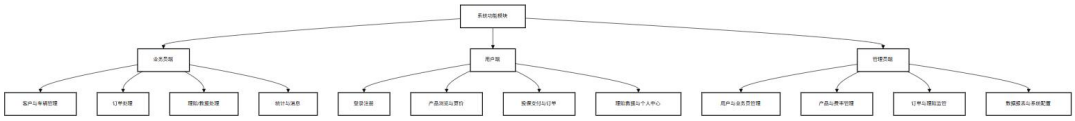

图 3-2 功能模块图

3.2.1 业务员功能模块

业务员功能模块核心是通过模块化架构实现需求落地，明确各子模块的设计逻辑、接口对接方式及模块间协同关系，结合功能模块图，确保设计可直接支撑后续开发，业务员功能模块图 3-3 如下。

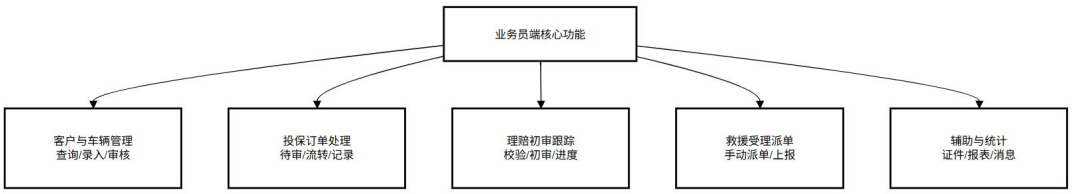

图 3-3 业务员功能模块图

具体设计如下：

(1) 客户与车辆管理模块：采用“接口对接+状态联动”设计，业务员通过客户查询接口检索用户信息，车辆信息录入与审核通过车辆接口完成；客户资料或车辆信息变更后触发同步接口，确保订单、理赔与救援模块使用的数据一致。

(2) 投保订单处理模块：采用“流程驱动+多状态联动”设计，用户端提交订单后即时同步至业务员端待审核列表；业务员审核通过/退回会触发状态同步接口，更新用户端订单状态；订单支付完成后状态自动流转为生效，异常订单需填写处理原因并记录在案。

(3) 理赔初审与进度跟踪模块：采用“材料校验+节点追踪”设计，理赔申请通过专用接口流转至业务员端初审；初审结果与意见同步至用户端与管理员端；理赔进度在各处理节点自动更新，保证流程可追溯。

(4) 救援受理与派单模块：设计救援申请受理接口与派单流程，支持手动派单与状态更新；救援进度通过接口实时同步至用户端与管理员端，异常情况可触发上报机制并通知相关人员。

(5) 证件与资料管理模块：对接证件上传与识别接口，支持证件核验、信息纠错与状态标注；资料变更同步至订单与理赔模块，避免信息不一致导致业务异常。

(6) 业务数据统计模块：设计多维度数据统计接口，自动采集订单量、理赔量、审核效率等指标，支持按日/周/月聚合，提供可视化图表与导出能力，为业务优化提供依据。

(7) 账户与消息管理模块：账号信息支持密码修改、手机号绑定与安全校验；消息提醒对接订单、理赔、救援等模块，自动推送待办与异常提示，保障业务处理及时性。

3.2.2 用户功能模块

用户功能模块聚焦“便捷性、易用性”设计，适配移动端操作场景，核心是通过简化交互流程、优化接口响应、实现模块联动，将用户需求转化为可落地的设计方案，结合功能模块图，如图 3-4 所示。

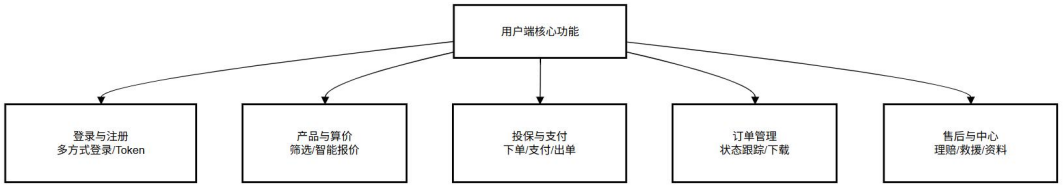

图 3-4 用户功能模块图

具体设计如下：

(1) 用户登录与注册模块：采用“多方式适配+安全便捷”设计，支持用户名/手机号+密码登录，注册流程精简并进行格式校验；登录会话通过 token 管理，避免重复登录。

(2) 产品浏览与在线算价模块：提供产品分类展示与筛选功能，算价模块对接智能报价接口，用户填写车辆信息与保障方案后即可获得保费预估，结果可直接进入投保流程。

(3) 投保下单与支付模块：用户选择产品与车辆后提交订单，支付接口支持即时支付与状态校验；订单创建后同步生成保单信息，订单状态实时更新。

(4) 订单与保单管理模块：提供订单查询、详情查看与状态跟踪，保单支持在线查看与下载；历史订单可按时间与状态筛选，便于用户管理。

(5) 理赔、救援与个人中心模块：理赔支持在线申请、材料上传与进度查询；救援支持位置提交与状态查看；个人中心整合车辆管理、证件管理、家庭成员信息复用与违章查询等功能，信息变更实时同步至相关业务模块。

3.2.3 管理员功能模块

管理员功能模块聚焦“规范化、可控性”设计，核心是通过权限分级、流程管控、数据联动，实现对用户、业务员、订单与理赔的全面监管，将监管需求转化为可落地的设计方案，结合功能模块图 3-5 所示，

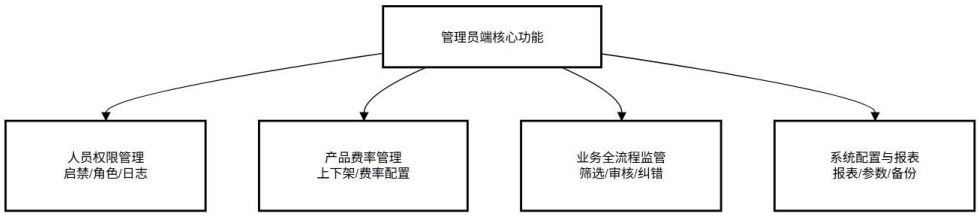

图 3-5 管理员功能模块图

具体设计如下：

(1) 用户与业务员管理模块：采用“权限分级管控”设计，管理员可查看用户与业务员信息、启用/禁用账号、重置密码，关键操作记录审计日志；业务员

权限可按岗位进行分配，确保职责清晰。

(2) 产品与费率规则管理模块：对接产品管理接口，支持产品新增、上下架、费率规则配置与版本管理；规则调整后自动同步至算价与订单模块，确保业务一致性。

(3) 订单与理赔监管模块：管理员可查看全量订单与理赔数据，支持多条件筛选与异常处理；理赔审核结果、退回原因与处理意见通过接口同步至用户端与业务员端，形成闭环管理。

(4) 系统数据与配置模块：运营数据通过多模块接口聚合采集，生成可视化报表并支持导出；系统基础参数与通知模板可在线配置，修改后即时生效；数据备份与恢复机制保障系统稳定性与数据安全。

3.3 数据库设计

3.3.1 概念结构设计

概念结构设计采用 E-R 模型（实体-关系模型），用于描述系统核心实体、实体属性及实体间的关联关系，不依赖具体数据库类型，聚焦“存储什么数据”，清晰梳理数据关联，为后续逻辑结构设计奠定基础。系统核心实体包括用户（User）、车辆（Vehicle）、保险产品（InsuranceProduct）、订单（Order）、理赔（Claim）、救援申请（RescueRequest）、用户证件（UserDocument）和家庭成员（FamilyMember）。主要关系为：用户与车辆一对多，用户与订单一对多，订单与理赔一对多，用户与救援申请一对多，用户与证件一对多，用户与家庭成员一对多，E-R 图 3-6 如下：

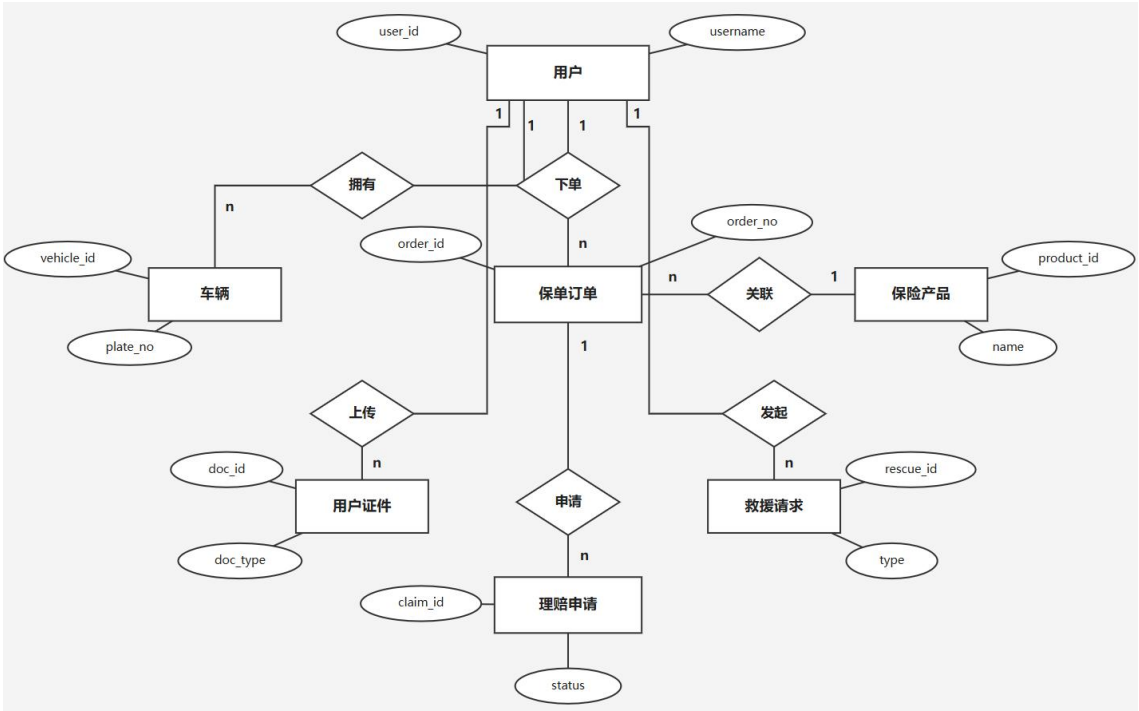

图 3-6 实体类图

结合车险系统的功能需求，梳理出以下核心实体，各实体说明如下：

（1）用户实体（USER）：对应系统用户，核心属性包括用户 ID、手机号、密码、姓名、头像、角色、账号状态、创建时间，用户实体属性具体如图 3-7 所示。

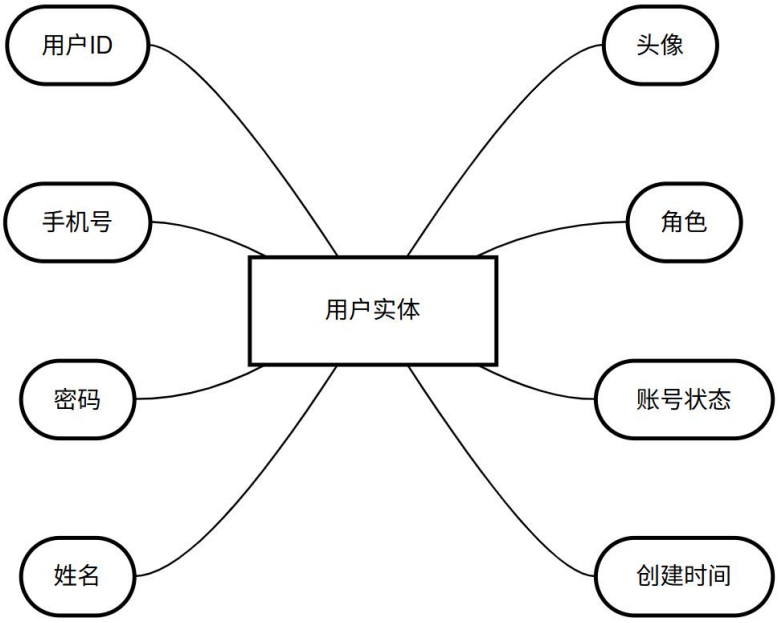

图 3-7 用户实体属性描述

（2）车辆实体（VEHICLE）：对应用户车辆，核心属性包括车辆 ID、用户 ID、车牌号、车架号（VIN）、品牌、车型、发动机号、注册日期，车辆实体属性具体如图 3-8 所示。

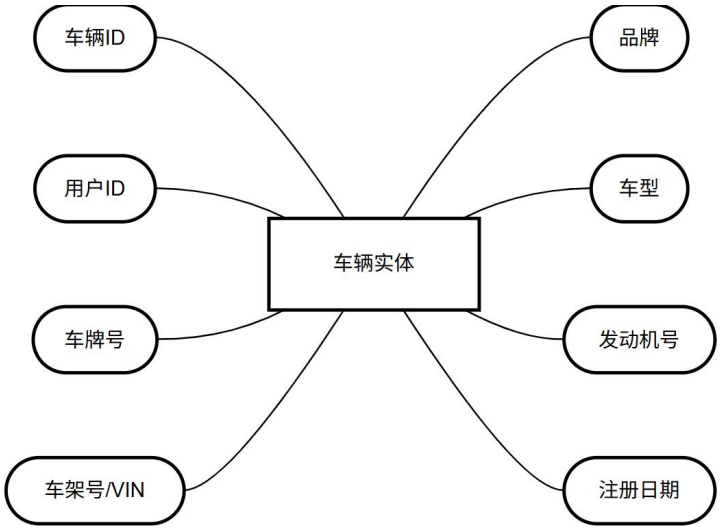

图 3-8 车辆实体属性描述

（3）保险产品实体（INSURANCE_PRODUCT）：对应险种产品，核心属性包括产品 ID、产品名称、保障范围、费率规则/基础费率、投保条件、状态、创建时间，保险产品实体属性具体如图 3-9 所示。

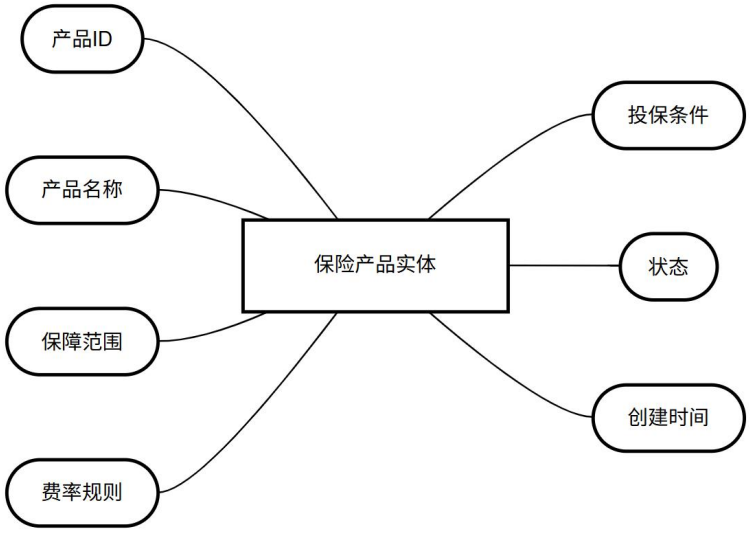

图 3-9 保险产品实体属性描述

（4）订单实体（ORDER）：对应投保订单，核心属性包括订单 ID、用户 ID、车辆 ID、产品 ID、保费金额、支付状态、订单状态、投保时间、保单号，订单实体属性具体如图 3-10 所示。

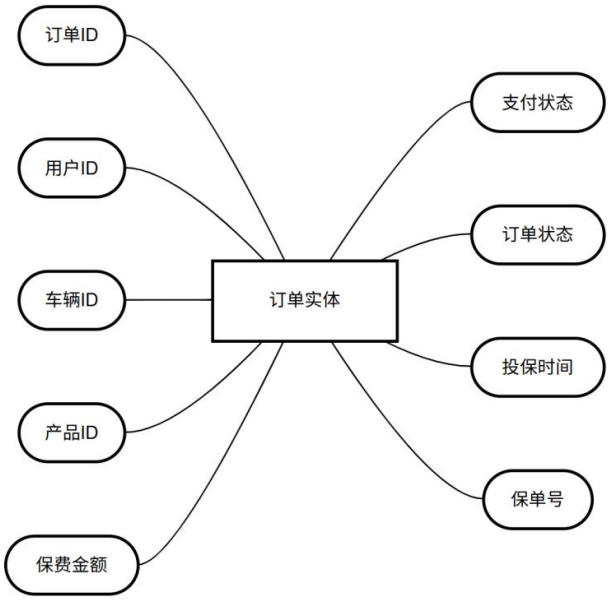

图 3-10 订单实体属性描述

（5）理赔实体（CLAIM）：对应理赔申请，核心属性包括理赔 ID、订单 ID、理赔类型、问题描述、理赔金额、审核状态、申请时间、处理时间，理赔实体属性具体如图 3-11 所示。

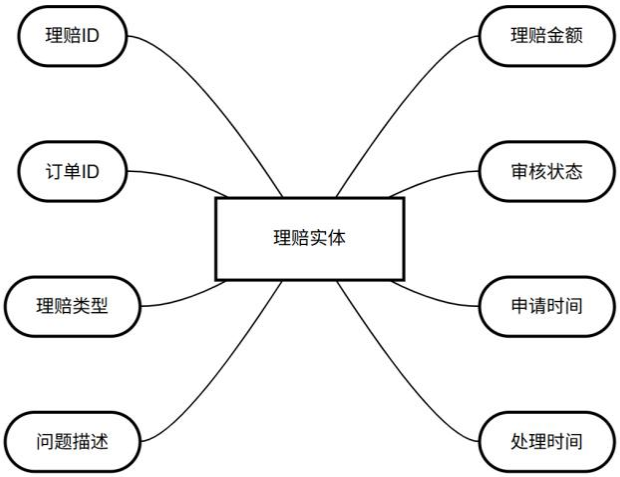

图 3-11 理赔实体属性描述。

（6）救援申请实体（RESCUE_REQUEST）：对应救援服务，核心属性包括救援 ID、用户 ID、救援类型、位置、申请时间、处理状态、完成时间，救援申请实体属性具体如图 3-12 所示。

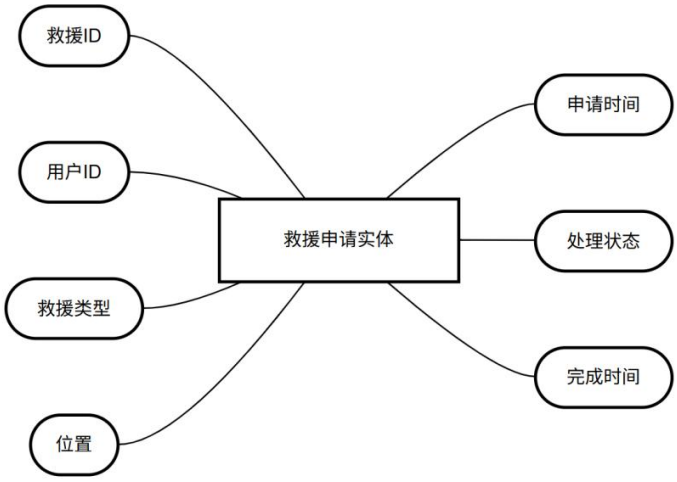

图 3-12 救援申请实体属性描述

（7）用户证件实体（USER_DOCUMENT）：对应证件资料，核心属性包括证件 ID、用户 ID、证件类型、文件地址、识别信息、状态、上传时间。用户证件实体属性具体如图 3-13 所示。

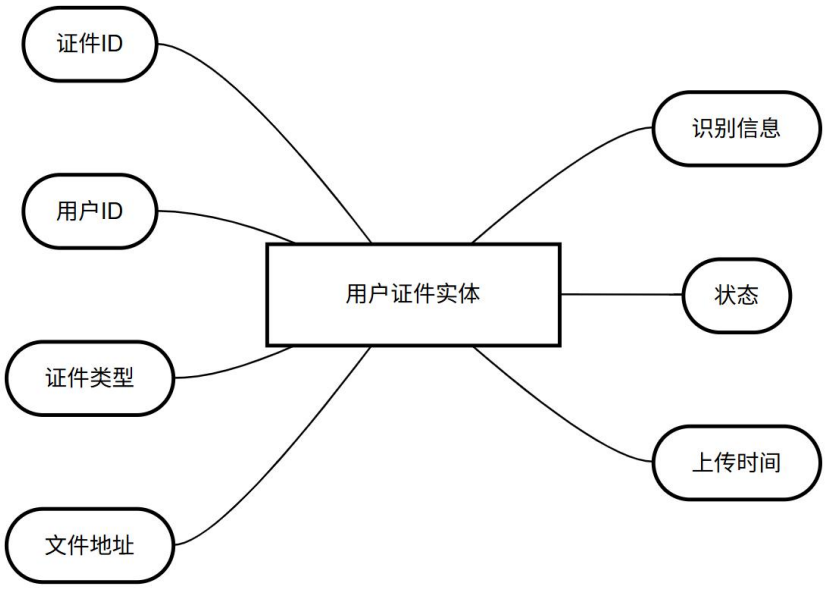

图 3-13 用户证件实体属性描述

（8）家庭成员实体（FAMILY_MEMBER）：对应家庭成员，核心属性包括成员 ID、用户 ID、姓名、关系、手机号、身份证号，家庭成员实体属性具体如图 3-14 所示。

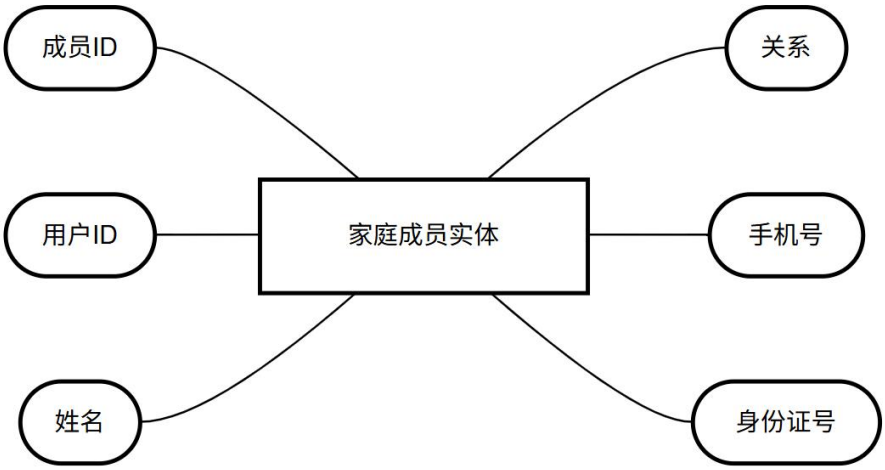

图 3-14 家庭成员实体属性描述

3.3.2 逻辑结构设计

数据库核心表包括 t_user、t_vehicle、t_insurance_product、t_order、t_claim、t_rescue_request、t_user_document、t_family_member。各表统一设计 create_time、update_time 和 deleted 字段，便于审计与维护。数据访问层通过 MyBatis-Plus 完成对象关系映射。

表 3-1 数据库结构

表名	表含义
t_user	用户表
t_vehicle	车辆表
t_insurance_product	保险产品表
t_product_rate_rule	产品费率规则版本表
t_order	订单表
t_claim	理赔表
t_rescue_request	救援申请表
t_user_document	用户证件表
t_family_member	家庭成员表

（1）用户表（t_user）

该表用于记录系统用户基础信息与账号状态，其属性包括 ID、用户名、密

码、真实姓名、手机号、邮箱、身份证号、角色、状态、头像、创建时间、更新时间、逻辑删除标记等信息，用户表的表结构如表 3-2 所示。

表 3-2 用户表结构

字段名	描述	数据类型	是否为空	约束
id	用户 ID	bigint	否	主键，自增
username	用户名	varchar(50)	否	唯一
password	密码	varchar(200)	否	-
real_name	真实姓名	varchar(50)	是	-
phone	手机号	varchar(20)	是	索引
email	邮箱	varchar(100)	是	-
id_card	身份证号	varchar(20)	是	-
role	角色	varchar(20)	是	默认 USER
status	状态	tinyint	是	默认 1
avatar	头像 URL	varchar(255)	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（2）车辆表（t_vehicle）

该表用于记录用户车辆信息及审核信息，其属性包括 ID、用户 ID、车牌号、品牌、车型、车架号（VIN）、发动机号、车辆类型、颜色、座位数、购买日期、注册日期、审核状态、审核原因、审核人 ID、审核时间、创建时间、更新时间、逻辑删除标记等信息，车辆表的表结构如表 3-3 所示。

表 3-3 车辆表结构

字段名	描述	数据类型	是否为空	约束
id	车辆 ID	bigint	否	主键，自增
user_id	用户 ID	bigint	否	索引
plate_no	车牌号	varchar(20)	否	索引
brand	品牌	varchar(50)	是	-
model	车型	varchar(50)	是	-
vin	车架号(VIN)	varchar(50)	是	-
engine_no	发动机号	varchar(50)	是	-

表 3-3 车辆表结构表（续表）

vehicle_type	车辆类型	varchar(20)	是	-
color	颜色	varchar(20)	是	-
seats	座位数	int	是	-
purchase_date	购买日期	date	是	-
register_date	注册日期	date	是	-
review_status	审核状态	varchar(20)	是	默认 PENDING
review_reason	审核原因	varchar(255)	是	-
reviewer_id	审核人 ID	bigint	是	-
review_time	审核时间	datetime	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（3）保险产品表（t_insurance_product）

该表用于记录保险产品信息，其属性包括 ID、产品名称、险种类型、基础价格、保额、产品描述、保障范围、适用车型、有效期、状态、排序、产品图标、创建时间、更新时间、逻辑删除标记等信息，保险产品表的表结构如表 3-4 所示。

表 3-4 保险产品表结构

字段名	描述	数据类型	是否为空	约束
id	产品 ID	bigint	否	主键，自增
name	产品名称	varchar(100)	否	-
type	险种类型	varchar(30)	否	索引
price	基础价格	decimal(10,2)	否	-
coverage	保额	decimal(15,2)	是	-
description	产品描述	text	是	-
coverage_scope	保障范围	text	是	-
applicable_vehicle	适用车型	varchar(200)	是	-
valid_months	有效期(月)	int	是	默认 12
status	状态	tinyint	是	默认 1
sort_order	排序	int	是	默认 0

表 3-4 保险产品表结构（续表）

icon	图标 URL	varchar(255)	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（4）订单表（t_order）

该表用于记录投保订单信息，其属性包括 ID、订单号、用户 ID、产品 ID、车辆 ID、保单号、总金额、实付金额、折扣金额、订单状态、支付方式、支付时间、生效日期、到期日期、投保人姓名、投保人电话、投保人身份证号、备注、创建时间、更新时间、逻辑删除标记等信息，订单表的表结构如表 3-5 所示。

表 3-5 订单表结构

字段名	描述	数据类型	是否为空	约束
id	订单 ID	bigint	否	主键，自增
order_no	订单号	varchar(50)	否	唯一，索引
user_id	用户 ID	bigint	否	索引
product_id	产品 ID	bigint	否	-
vehicle_id	车辆 ID	bigint	否	-
policy_no	保单号	varchar(50)	是	-
total_amount	总金额	decimal(10,2)	否	-
paid_amount	实付金额	decimal(10,2)	是	-
discount_amount	折扣金额	decimal(10,2)	是	默认 0
status	订单状态	varchar(20)	是	默认 PENDING
payment_method	支付方式	varchar(20)	是	-
payment_time	支付时间	datetime	是	-
effective_date	生效日期	date	是	-
expiry_date	到期日期	date	是	-
insured_name	投保人姓名	varchar(50)	是	-
insured_phone	投保人电话	varchar(20)	是	-
insured_id_card	投保人身份证	varchar(20)	是	-

表 3-5 订单表结构（续表）

remark	备注	text	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（5）理赔表（t_claim）

该表用于记录理赔申请及处理过程信息，其属性包括 ID、理赔单号、订单 ID、用户 ID、理赔金额、理赔类型、理赔状态、事故描述、事故地点、事故时间、证据材料、审核意见、审核人 ID、审核时间、赔付时间、创建时间、更新时间、逻辑删除标记等信息，理赔表的表结构如表 3-6 所示。

表 3-6 理赔表结构

字段名	描述	数据类型	是否为空	约束
id	理赔 ID	bigint	否	主键，自增
claim_no	理赔单号	varchar(50)	否	唯一，索引
order_id	订单 ID	bigint	否	索引
user_id	用户 ID	bigint	否	索引
amount	理赔金额	decimal(10,2)	是	-
claim_type	理赔类型	varchar(20)	是	-
status	理赔状态	varchar(20)	是	SUBMITTED
description	事故描述	text	是	-
accident_location	事故地点	varchar(200)	是	-
accident_time	事故时间	datetime	是	-
evidence_materials	证据材料	text	是	-
review_comment	审核意见	text	是	-
reviewer_id	审核人 ID	bigint	是	-
review_time	审核时间	datetime	是	-
paid_time	赔付时间	datetime	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（6）救援申请表（t_rescue_request）

该表用于记录道路救援申请信息，其属性包括 ID、用户 ID、车辆 ID、救援类型、处理状态、位置描述、经度、纬度、详细描述、联系电话、创建时间、更新时间、逻辑删除标记等信息，救援申请表的表结构如表 3-7 所示。

表 3-7 救援申请表结构

字段名	描述	数据类型	是否为空	约束
user_id	用户 ID	bigint	否	索引
vehicle_id	车辆 ID	bigint	是	-
type	救援类型	varchar(20)	否	-
status	处理状态	varchar(20)	是	默认 PENDING
location	位置描述	varchar(255)	是	-
longitude	经度	double	是	-
latitude	纬度	double	是	-
description	详细描述	text	是	-
contact_phone	联系电话	varchar(20)	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（7）用户证件表（t_user_document）

该表用于记录用户证件与电子保单信息，其属性包括 ID、用户 ID、证件类型、证件号码、所有人姓名、到期日期、证件图片路径、创建时间、更新时间、逻辑删除标记等信息，用户证件表的表结构如表 3-8 所示。

表 3-8 用户证件表结构

字段名	描述	数据类型	是否为空	约束
id	证件 ID	bigint	否	主键，自增
user_id	用户 ID	bigint	否	索引
doc_type	证件类型	varchar(20)	否	-
doc_no	证件号码	varchar(50)	是	-
owner_name	所有人姓名	varchar(50)	是	-

表 3-8 用户证件表结构（续表）

expiry_date	到期日期	date	是	-
image_url	证件图片	varchar(255)	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

（8）家庭成员表（t_family_member）

该表用于记录用户家庭成员信息，其属性包括 ID、用户 ID、姓名、关系、身份证号、手机号、创建时间、更新时间、逻辑删除标记等信息，家庭成员表的表结构如表 3-9 所示。

表 3-9 家庭成员表结构

字段名	描述	数据类型	是否为空	约束
id	主键 ID	bigint	否	主键，自增
user_id	用户 ID	bigint	否	索引
name	姓名	varchar(50)	否	-
relation	关系	varchar(20)	是	-
id_card	身份证号	varchar(20)	是	-
phone	手机号	varchar(20)	是	-
create_time	创建时间	datetime	是	默认当前时间
update_time	更新时间	datetime	是	自动更新
deleted	逻辑删除	tinyint	是	默认 0

4 详细设计与实现

4.1 系统功能设计

对系统各端功能进行进一步细化拆解，明确每个功能点的详细设计要求、交互流程、输入输出规范及异常处理逻辑，同时确定功能设计的优先级，确保功能设计贴合实际使用场景、可落地性强。本系统功能设计围绕业务员端、用户端、管理员端三大核心端展开，严格对应前文需求分析与概要设计的模块划分，兼顾功能完整性、易用性与可扩展性，每个功能点均明确设计目标、操作流程与约束条件，为后续系统功能实现提供清晰、具体的设计依据

4.1.1 业务员端功能详细设计

（1）投保订单审核与状态流转功能

业务员在订单列表中查看订单详情，完成产品信息、车辆信息与证件资料的一致性校对后进行审核通过或退回补充。审核完成后系统自动同步订单状态至用户端，确保订单在待审核、待支付、生效等环节中的状态保持一致，提升业务处理效率与流程可控性。具体流程如图 4-1 所示。

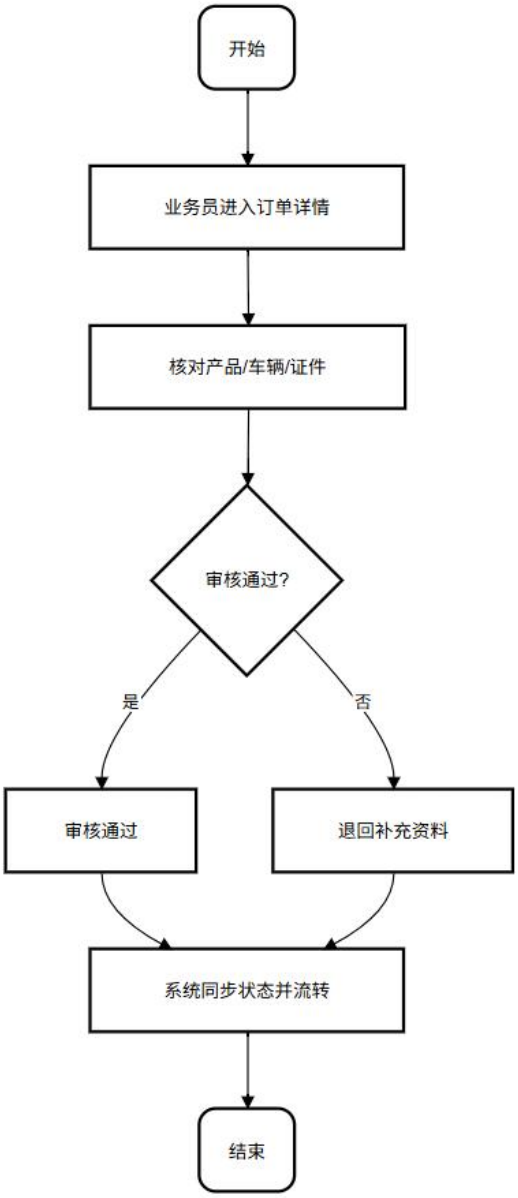

图 4-1 投保订单审核与状态流转流程

（2）理赔初审与进度跟踪功能

业务员在理赔列表中查看申请材料并完成初审处理，初审结果提交后进入节点跟进阶段，业务员可持续更新理赔处理进度。该功能用于保障理赔流程透明可

追溯，使用户能够及时掌握处理状态，同时便于业务员开展后续协同与反馈，具体流程如图 4-2 所示。

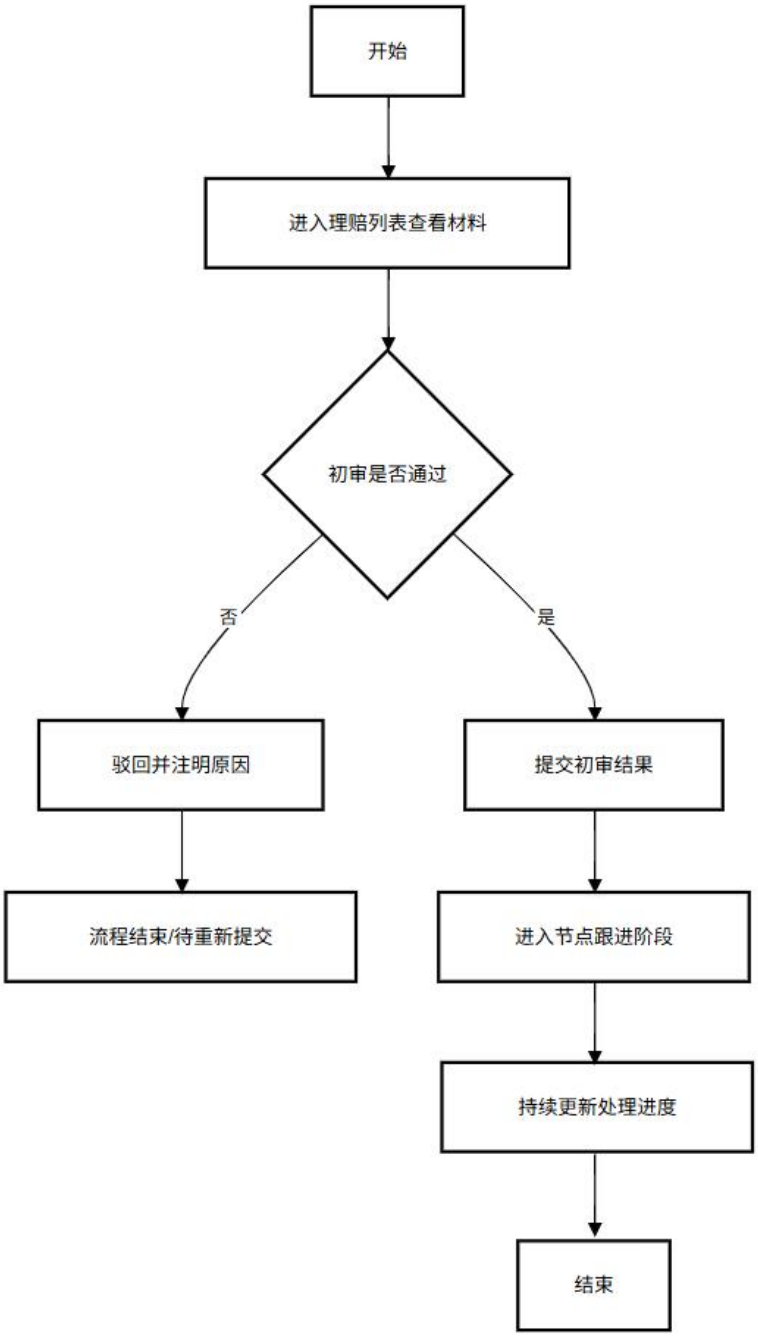

图 4-2 理赔初审与进度跟踪流程

4.1.2 用户端功能详细设计

（1）投保下单与在线支付功能

用户在产品列表选择险种后，结合车辆信息完成投保信息确认并提交订单，随后进入支付环节完成在线支付。支付成功后系统自动生成保单并同步更新订单

状态，使用户能够在订单中心直接查看投保结果，形成从投保到出单的完整业务闭环，具体流程如图 4-3 所示。

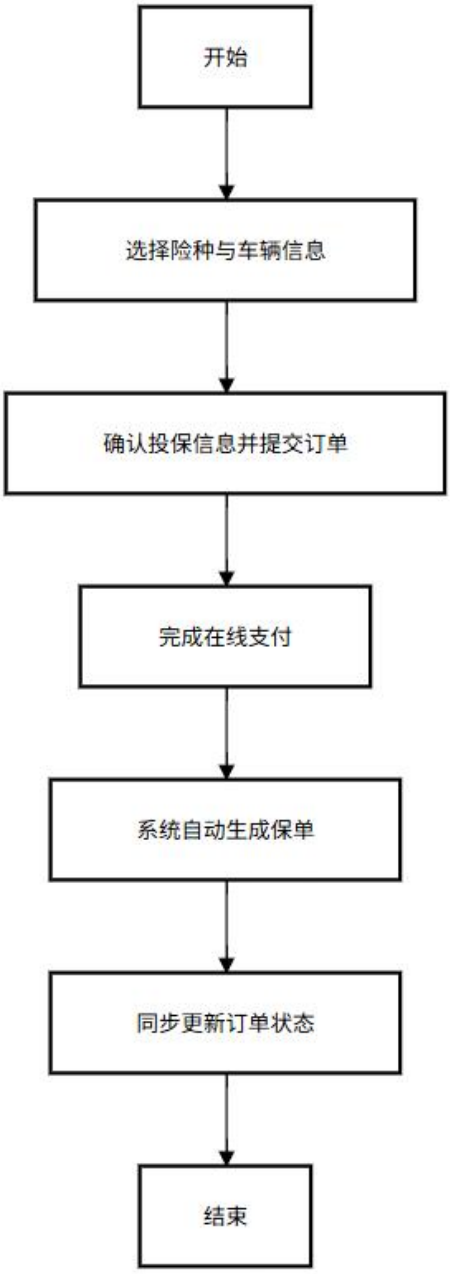

图 4-3 投保下单与在线支付流程

（2）理赔申请与进度查询功能

用户在理赔模块选择关联订单，填写事故信息并上传相关材料后提交申请。提交成功后，系统按处理节点持续更新理赔状态，用户可在理赔详情中实时查看审核进度与处理结果，从而提升理赔流程透明度与使用体验，具体流程如图 4-4 所示。

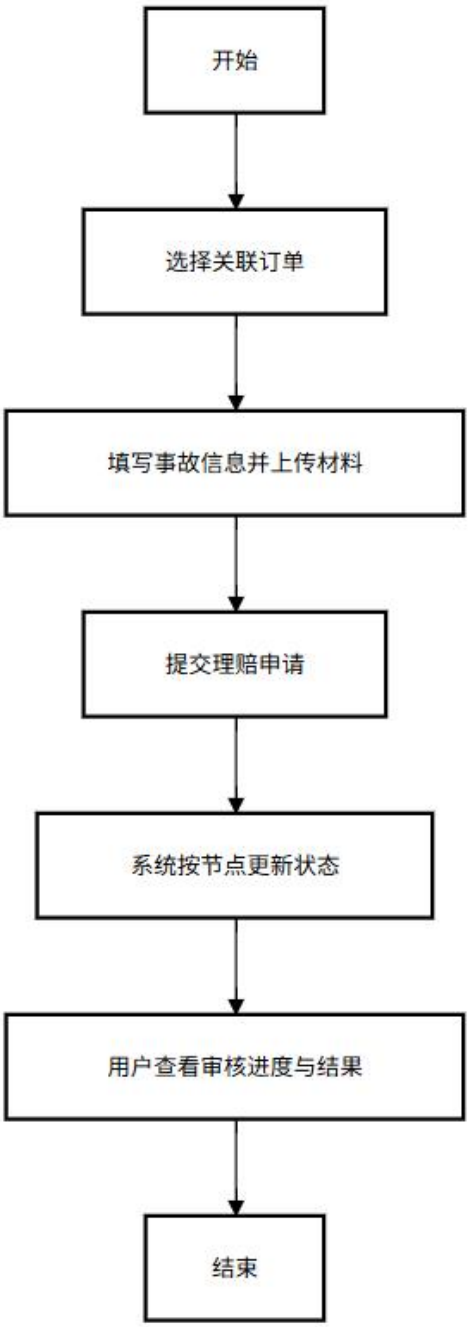

图 4-4 理赔申请与进度查询流程

（3）救援申请与辅助服务功能

用户在救援模块提交当前位置并选择救援类型后，系统进入受理流程并返回处理状态。与此同时，用户端提供违章查询、证件管理、家庭成员管理等辅助能力，用于满足高频服务场景，提升用车全周期的服务便利性。

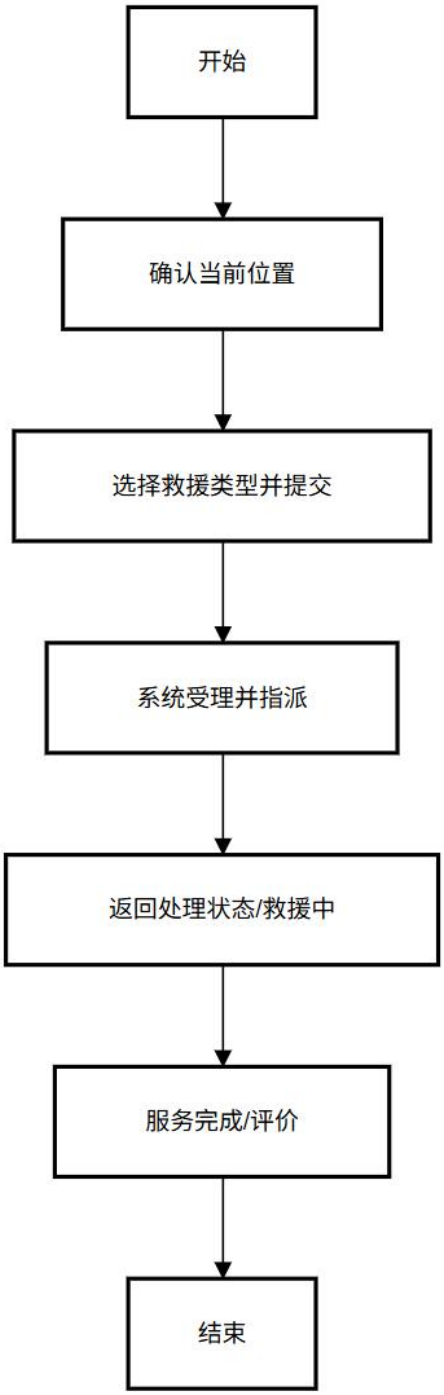

图 4-5 救援申请与辅助服务流程

4.1.3 管理员端功能详细设计

（1）用户与业务员账号管理功能

管理员可在账号管理模块统一查看用户与业务员信息，并执行启用、禁用和权限配置操作。该功能用于保障角色权限边界清晰、账号状态可控，为系统稳定运行提供基础管理支撑，具体流程如图 4-6 所示。

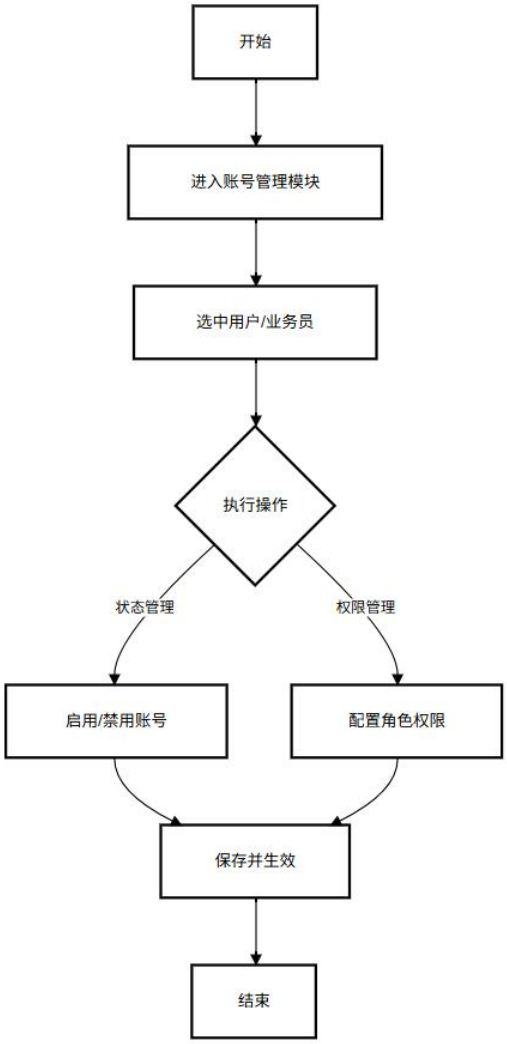

图 4-6 用户与业务员账号管理流程

（2）产品与费率规则管理功能

管理员在产品管理模块完成产品新增、编辑、上下架及费率规则维护，规则调整后可同步至算价与订单相关业务。该功能用于保证产品数据与费率策略统一生效，提升平台运营管理的规范性。具体流程图如图 4-7 所示。

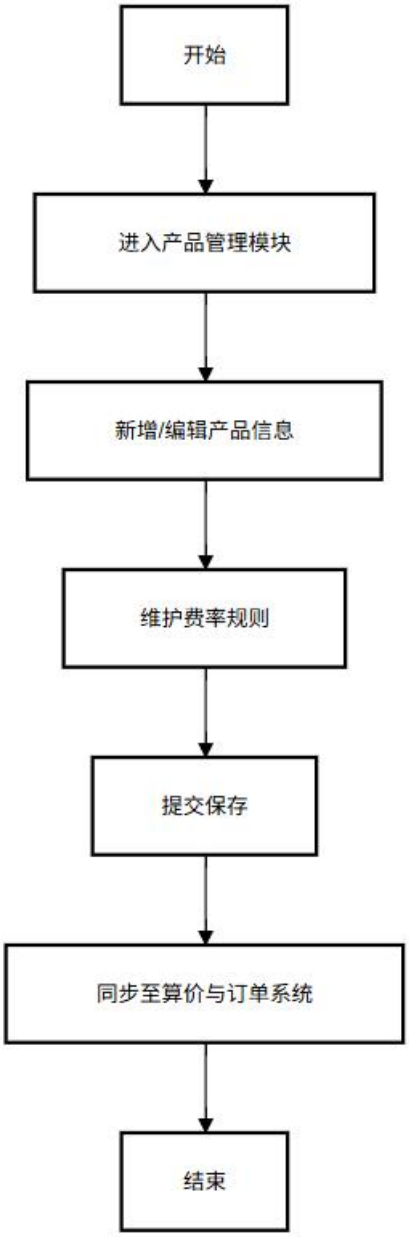

图 4-7 产品与费率规则管理流程

（3）订单理赔监管与系统配置功能

管理员可对订单与理赔进行全量查看、筛选与异常处理，同时可在系统配置模块完成报表导出、参数维护与通知模板管理。该功能将业务监管与系统治理结合，提升平台可控性、可追溯性与持续运维能力。具体流程图如图 4-8 所示。

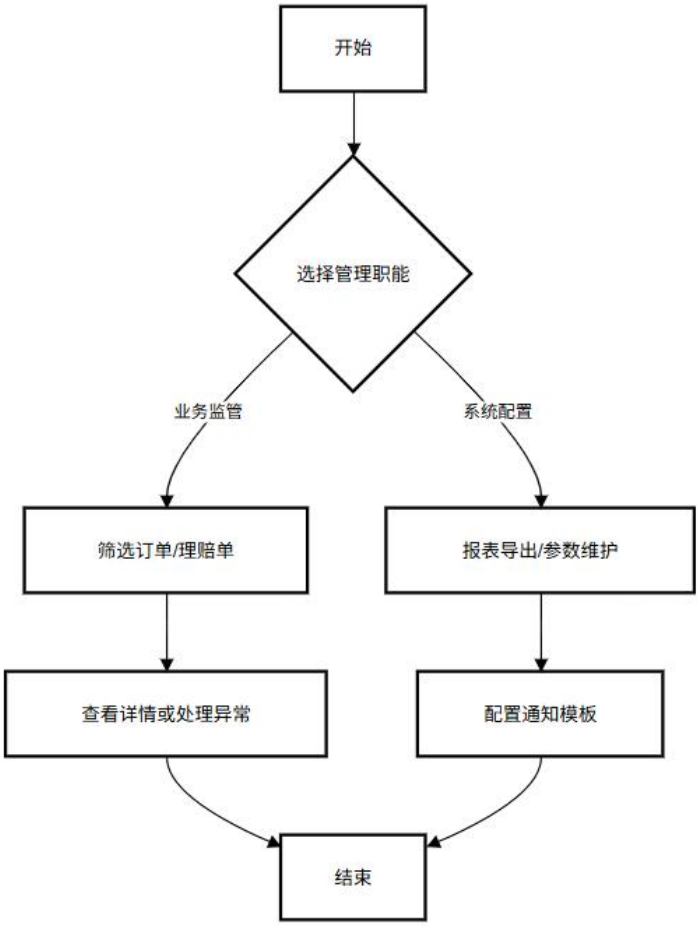

图 4-8 订单理赔监管与系统配置流程

4..2 系统功能实现

4.2.1 用户功能的设计与实现

（1）投保下单与在线支付功能实现

投保下单与支付功能由产品查询、订单创建与支付确认三个环节组成。用户进入投保页面后，前端通过 /api/products/list 获取可投保产品清单，并通过 /api/products/{id} 查看保障范围、价格与适用条件，完成险种选择。确认产品与车辆后，前端调用 /api/orders/create 提交下单请求，后端在校验产品状态、车辆信息后生成订单号并写入订单数据。

订单创建时系统会结合产品信息与费率规则进行保费计算，并写入投保人姓名、手机号、证件号等核心字段，初始化状态为待处理。用户在支付阶段调用 /api/orders/pay/{id} 完成支付，后端同步更新支付方式、支付时间、实付金额、保单号、生效日期和到期日期，并将电子保单信息写入证件表，形成“产品选择—订单创建—在线支付—保单生效”的完整闭环。该设计保证了订单主链路状态

清晰、数据可追踪、多端展示一致。

支付功能图如图 4-9 所示。

图 4-9 支付功能图

投保订单确认图如图 4-10 所示。

图 4-10 投保订单确认图

（2）理赔申请功能实现

理赔申请功能由 ClaimsController 提供，核心接口为 /api/claims/submit（POST）。用户在前端填写理赔金额、事故描述、事故地点、事故时间和证据材料后提交申请。后端接收 ClaimSubmitRequest，先校验 orderId 与 userId 等关键字段，再校验关联订单是否存在。校验通过后，系统自动生成 claimNo，并将理赔记录以 SUBMITTED 状态写入 t_claim 表。这样形成“用户提交—后端校验—理赔入库”的完整流程，便于后续审核处理。理赔申请功能图如图 4-12 所示。

图 4-12 理赔申请功能图

（3）救援申请功能实现

救援申请功能由 RescueController 实现。用户进入救援页面后，先调用 /api/rescue/types（GET）获取救援类型，再填写位置、联系方式、车辆信息与问题描述，通过 /api/rescue/submit（POST）提交。后端接收 RescueSubmitRequest 并完成参数校验后，将请求写入 t_rescue_request，默认状态设为 PENDING。入库字段包括 vehicleId、location、longitude、latitude、description、contactPhone，可直接用于后续派单。该功能实现“类型选择—在线提交—状态入库”的快速链路。救援申请功能图如图 4-15 所示。

图 4-15 救援申请功能图

（4）用户登录功能实现

用户登录功能由 AuthController 的 /api/auth/login（POST）提供。前端提交账号与密码后，后端按“用户名或手机号”查询用户，并校验密码是否匹配。若账号不存在、密码错误或账号状态为禁用，系统直接返回业务异常；校验成功后生成 JWT 并返回前端。返回内容包含 token 和脱敏后的用户信息，供前端保存登录态并访问后续接口。用户登录功能图如图 4-21 所示。

图 4-21 用户登录功能图

（5）车辆信息维护功能实现

车辆信息维护功能在 VehicleController 中实现，接口前缀为 /api/vehicles。用户首次录入车辆时调用 /api/vehicles/create（POST），未传 reviewStatus 时系统自动设为 PENDING。需要修改时调用 /api/vehicles/update/{id}（PUT），可维护车牌号、品牌、车型、VIN、发动机号、购置日期、登记日期等字段。后端会先按 id 检查记录是否存在，再执行更新。该功能形成“新增建档—信息修改—数据保存”的单功能流程，车辆信息维护功能图如图 4-22 所示。

图 4-22 车辆信息维护功能图

（6）违章查询功能实现

违章查询功能由 ViolationController 提供，接口地址为 /api/violation/query（GET）。用户输入车牌号 plateNo 后，前端发起查询请求。后端返回结构化结果，包括 count、latestDate、location、description、fine 等字段，前端可直接展示违章摘要。该功能实现“车牌输入—结果返回”的快速查询链路，满足用户日常自助查询需求，违章查询功能图如图 4-23 所示。

图 4-23 违章查询功能图

4.2.2 业务人员的功能设计与实现

（1）投保订单审核功能实现

投保订单审核功能由 OrdersController 提供。服务人员通过 /api/orders/list 获取订单列表，并可按 userId、status 筛选目标订单。进入详情页后，前端调用 /api/orders/{id} 查询订单明细，服务人员核对投保信息、车辆信息和证件资料后，通过 /api/orders/update/{id} 提交审核结果。后端支持回写 status、remark、discountAmount，实现“列表筛选—详情核验—审核回写”闭环，订单审核功能图如图 4-18 所示。

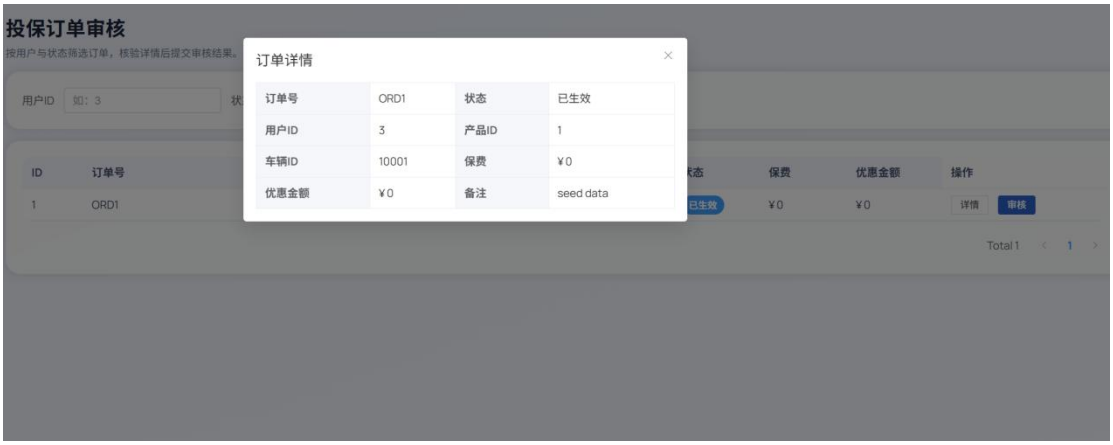

图 4-18 订单审核功能图

（2）理赔初审功能实现

理赔初审功能由 ClaimsController 实现。服务人员先通过 /api/claims/list 获取理赔申请，再通过 /api/claims/{id} 查看单条理赔详情。审核时调用 /api/claims/{id}/review，后端回写 status、reviewComment、reviewerId、reviewTime。当审核状态为 APPROVED 时，系统同步写入 paidTime。该功能形成“记录查看—审核判定—结果留痕”的单功能链路，理赔初审功能图如图 4-21 所示。

图 4-21 理赔初审功能图

（3）客户车辆核验功能实现

客户车辆核验功能依托 VehicleController 完成。服务人员通过 /api/vehicles/list 按 userId 查询车辆档案，核对车牌号、VIN、发动机号和登记日期等关键字段。发现问题时可指导用户通过 /api/vehicles/update/{id} 补充信息，并通过 /api/vehicles/review/{id} 回写核验结果。后端记录 reviewStatus、reviewReason、reviewerId、reviewTime，保证核验过程可追踪，车辆核验功能截图如图 4-24 所示。

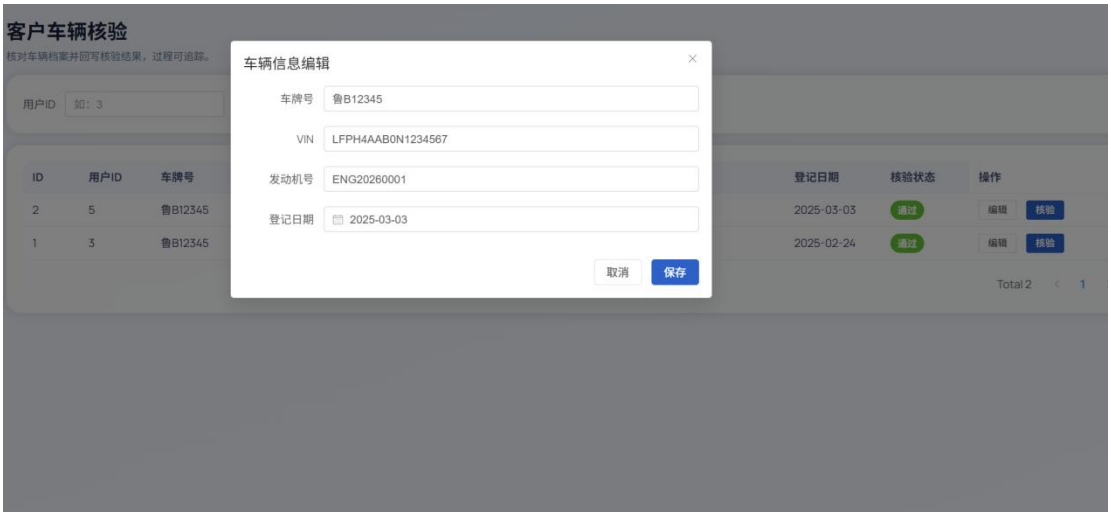

图 4-24 车辆核验功能截图

（4）产品信息检索功能实现

产品检索功能由 ProductsController 提供。业务人员通过 /api/products/list（GET）按 status 筛选产品，系统按 sortOrder 升序、id 降序返回结果。查看单产品时调用 /api/products/{id}（GET），可获取名称、类型、价格、保障范围、适用车型、有效期等字段。该功能满足咨询场景下的快速检索与信息核对需求，产品检索功能截图如图 4-24 所示。

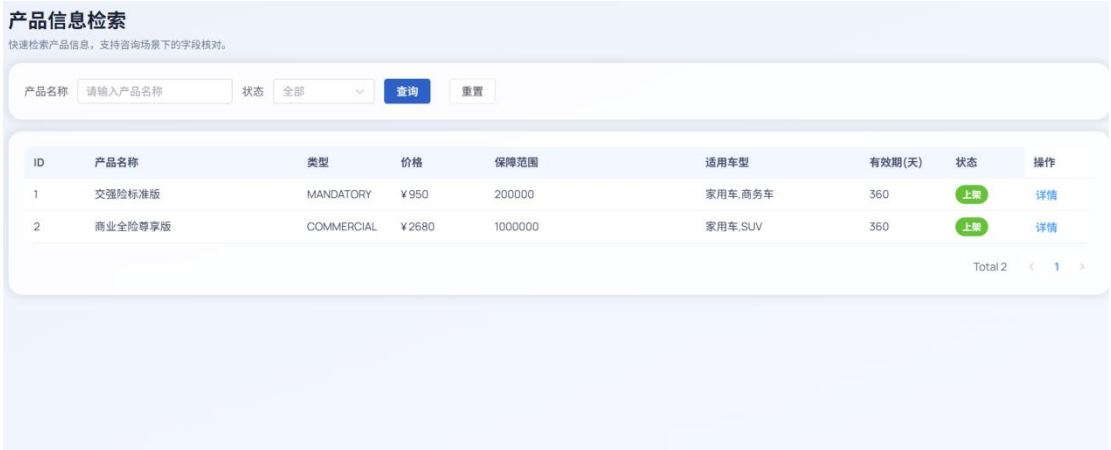

图 4-24 产品检索功能截图

（5）费率规则查询功能实现

费率规则查询功能由 RateRulesController 提供。业务人员通过 /api/rate-rules/list（GET）按 productId、status 查询规则列表。点击单条规则时调用 /api/rate-rules/{id}（GET），可查看 versionNo、baseRate、ageFactor、accidentFactor 等关键参数。该功能用于报价前参数核对，减少人工口径偏差，费率规则查询功能截图如图 4-25 所示。

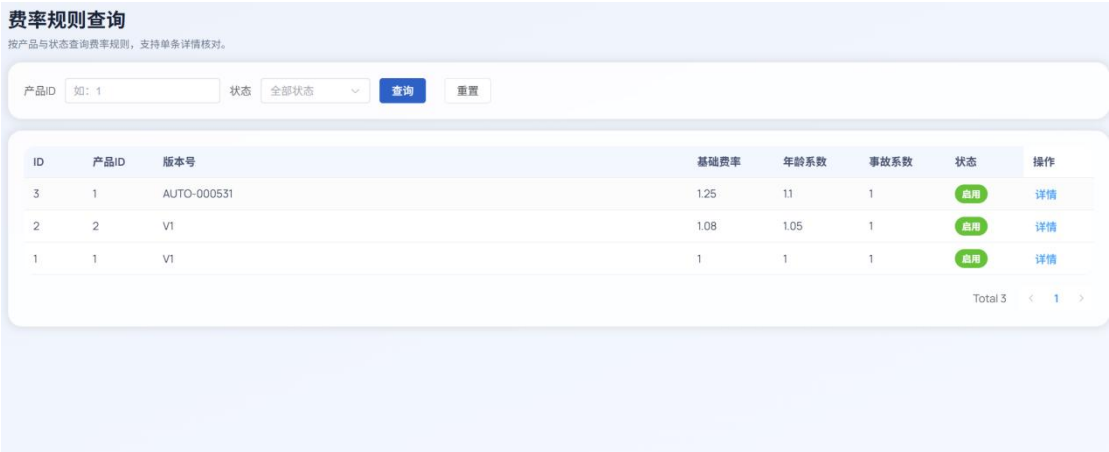

图 4-25 费率规则查询功能截图

（6）配置参数查询功能实现

配置参数查询功能由 SystemConfigController 实现。业务人员通过 /api/system-config/list（GET）按 category、status 过滤配置项。系统按分类和键名升序返回结果，便于快速定位所需参数；查看单项详情可调用 /api/system-config/{id}（GET）。该功能用于统一前线解释口径，避免同类问题出现多版本说法，配置参数查询功能图如图 4-26 所示。

图 4-26 配置参数查询功能图

4.2.3 管理员功能的设计与实现

（1）用户与服务人员账号管理功能实现

账号管理功能由 UsersController 提供。管理员通过 /api/users/list（GET）按用户名、手机号、真实姓名检索账户，并按角色筛选。查看详情使用 /api/users/{id}（GET），更新资料使用 /api/users/update/{id}（PUT），可维护姓名、手机号、邮箱、证件号、角色、状态和头像等字段。返回时统一脱敏密码，保障数据安全，用户与服务人员账号管理功能图如图 4-16 所示。

图 4-16 用户与服务人员账号管理功能图

（2）产品管理功能实现

产品管理功能由 ProductsController 实现。管理员可通过 /api/products/create 新增产品，通过 /api/products/update/{id} 修改产品，通过 /api/products/delete/{id} 下线产品。可维护字段包括名称、类型、价格、保障范围、适用车型、有效期、状态与排序。该功能实现产品全生命周期管理，保证前台展示与后台策略一致，产品管理功能图如图 4-17 所示。

图 4-17 产品管理功能图

（3）订单监管功能实现

订单监管功能由 OrdersController 提供。管理员通过 /api/orders/list 全量查看订单，并可按用户和状态筛选问题订单。通过 /api/orders/{id} 查看订单详情后，可结合更新接口进行人工干预和备注补充。该功能用于异常订单排查与状态纠偏，提升运营可控性，订单监管功能图如图 4-18 所示。

图 4-18 订单监管功能图

（4）费率规则发布功能实现

费率规则发布功能由 RateRulesController 承载。管理员通过 /api/rate-rules/create 新建规则，系统会校验 productId、versionNo 非空，且费率系数必须大于 0。通过 /api/rate-rules/update/{id} 可维护版本和系数，通过 /api/rate-rules/delete/{id} 清理废弃规则。该功能保证费率版本可发布、可维护、可回收，费率规则发布功能图如图 4-27 所示。

图 4-27 费率规则发布功能图

（5）系统配置新增功能实现

系统配置新增功能由 SystemConfigController 提供，接口为 /api/system-config/create（POST）。管理员新增配置时，后端先校验 configKey 非空，再检查键是否重复。若配置键重复则返回业务异常，未传状态时默认启用。该功能用于保证配置项新增阶段的合法性与唯一性，减少后续配置冲突，系统配置新增功能图如图 4-28 所示。

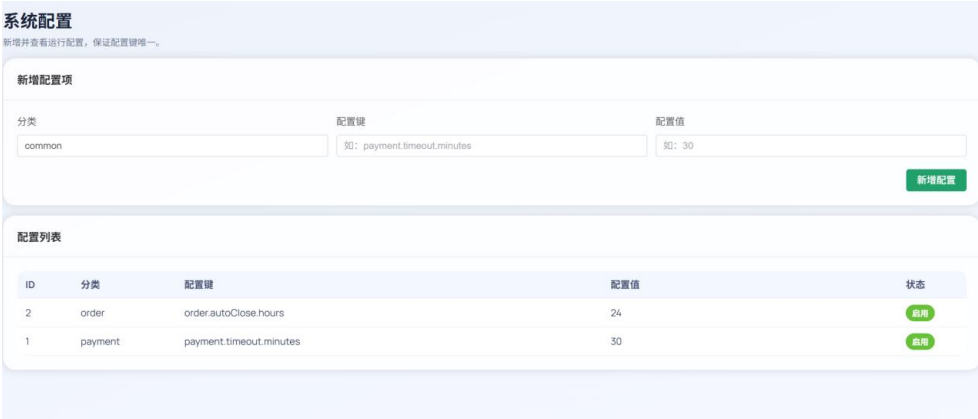

图 4-28 系统配置新增功能图

（6）账号状态管控功能实现

账号状态管控功能由 UsersController 的 /api/users/status/{id}（PUT）实现。

管理员可对账号执行启用或禁用，状态更新后立即生效。当账号被禁用时，登录流程会拒绝签发令牌，已受保护接口也会因鉴权失败被拦截。该功能实现“状态设置—登录限制—接口拦截”的安全闭环，账号状态管控功能图如图 4-29 所示。

图 4-29 账号状态管控功能图

5 系统测试

5.1 订单创建功能测试

在小程序端选择产品和车辆并填写投保人信息后提交订单，用于验证订单创建、状态初始化与列表展示是否正确。其测试用例如表 5-1 所示。

表 5-1 订单创建功能测试用例

编号	测试内容	执行操作	预期结果	测试结果
1	正常创建订单	登录小程序，选择保险产品和车辆，填写投保人信息后提交订单	系统成功生成订单号并写入订单记录，订单状态为 PENDING	符合预期
2	未选择车辆提交	在订单创建页不选择车辆直接提交	页面提示“请选择车辆”，不创建订单	符合预期
3	创建后列表验证	订单提交成功后进入订单列表页查询最新记录	可查询到新订单，状态与详情页信息一致	符合预期

5.2 理赔申请功能测试

用户在理赔模块提交申请后，系统应正确生成理赔记录并支持后续审核流转。其测试用例如表 5-2 所示。

表 5-2 理赔申请功能测试用例

编号	测试内容	执行操作	预期结果	测试结果
1	正常提交理赔	选择有效订单，填写事故信息并上传材料后提交	系统生成理赔单号，初始状为 SUBMITTED	符合预期
2	申请信息不完整	不填写必填事故信息直接提交	页面提示补全信息，理赔申请不提交	符合预期
3	审核状态流转	管理端对理赔单执行审核通过/驳回操作	理赔状态由 SUBMITTED 至 REVIEWING、APPROVED 或 REJECTED	符合预期

5.3 救援申请功能测试

用户提交救援请求后，系统应正确保存申请并支持记录查询。其测试用例如表 5-3 所示。

表 5-3 救援申请功能测试用例

编号	测试内容	执行操作	预期结果	测试结果
1	正常提交救援	填写救援位置、联系电话和救援类型后提交	系统成功写入救援记录并返回受理结果	符合预期
2	定位失败处理	定位失败后手动确认地址并提交	系统可继续提交请求，页面提示并进入受理流程不创建订单	符合预期
3	救援记录查询	进入救援历史页面查看记录	可查询到已提交救援记录，状态显示正常	符合预期

5.4 智能算价功能测试

智能算价功能用于验证不同险种与参数组合下的保费计算准确性，测试采用后端接口与单元测试结合方式进行。其测试用例如表 5-4 所示。

表 5-4 智能算价功能测试用例

编号	测试内容	执行操作	预期结果	测试结果
1	交强险基础算价	输入标准车辆参数并选择交强险进行算价	返回保费结果，金额计算正确且可展示	符合预期
2	附加险组合算价	选择第三者责任险、车损险等组合方案进行算价	返回组合保费，计算结果与规则一致	符合预期
3	参数异常校验	提交缺失或非法参数进行算价	系统返回参数错误提示，不生成无效报价	符合预期

本次系统测试覆盖了订单、理赔、救援及智能算价等核心业务流程，测试结果均符合预期，系统主要功能运行稳定，可满足当前业务使用需求。

6 总结

本文围绕车险销售与服务场景，完成了基于微信小程序的车辆保险服务平台的设计与实现。系统采用前后端分离架构，后端基于 Spring Boot 构建业务服务，管理端基于 Vue3 实现运营与监管功能，用户端基于微信小程序提供移动化业务办理入口，并结合 MySQL 完成核心业务数据的持久化存储，从而形成“后端服务 + 管理后台 + 小程序端”三端协同的整体解决方案。在项目实现过程中，本文按照需求分析、概要设计、详细设计与系统测试的思路逐步推进，完成了用户管理、车辆管理、保险产品管理、在线投保下单、订单支付、保单管理、理赔申请、道路救援、证件资料管理、家庭成员管理以及违章查询等核心功能模块的设计与实现。同时，系统围绕订单主链路构建了较完整的业务闭环，在用户完成投保与支付后，系统能够联动生成电子保单并同步更新相关业务状态，提高了订单处理效率与数据一致性；在管理端则实现了对订单、产品、理赔与用户信息的集中管理与监管，增强了平台的可维护性和业务支撑能力。整体来看，本文所实现的系统已经能够满足毕业设计阶段对车险服务平台的主要功能需求。当然，

受开发周期、真实业务环境和第三方接口接入条件限制，系统目前仍有进一步完善的空间。后续可以从以下几个方面继续优化：一是接入真实 OCR 识别、支付网关、违章查询与救援调度等外部服务，进一步提升系统在真实业务场景中的可用性；二是细化角色权限控制、日志审计与异常告警机制，增强系统安全性与可追溯性；三是完善接口自动化测试、性能测试与高并发场景下的稳定性验证，提高系统可靠性；四是继续优化算价逻辑、数据统计分析与多端协同能力，提升平台的智能化和扩展能力。总体而言，本文完成的车辆保险服务平台具有一定的实际应用价值，也为后续同类车险业务系统的研究与实现提供了参考。

参考文献

[1] 张一飞, 左涛. 基于微信小程序云开发测绘项目信息管理系统[J]. 测绘技术装备, 2021, 23(1):84-87.

[2] 李洋. SSM 框架在 Web 应用开发中的设计与实现[J]. 计算机技术与发展, 2016, 26(12):190-194.

[3] 朱志慧, 蔡洁. 基于 SpringBoot+Vue+Uni-app 框架的校园失物招领系统[J]. 电子技术与软件工程, 2022(17):66-69.

[4] 霍福华, 韩慧. 基于 SpringBoot 微服务架构下前后端分离的 MVVM 模型[J]. 电子技术与软件工程, 2022(1):77-80.

[5]陈帅. 微信点餐系统小程序的设计与实现[J]. 电子技术与软件工程, 2021(24):36-37.

[6] 黄琳, 林国钦. 基于 Java Web 的宠物店管理系统的设计[J]. 电子技术与软件工程, 2019(4):52-53.

[7] 王宇, 夏丽丽, 何颖刚. 基于 SSM 框架的高校毕业论文管理系统的设计与实现[J]. 电子技术与软件工程, 2021(22):32-35.

[8] 郭丽蓉. 基于 Python 的网络爬虫程序设计[J]. 电子技术与软件工程, 2017(23):248-249.

[9] 杨锦山, 王辉. 基于 SpringBoot 的项目信息管理系统的设计与实现[J]. 电子技术与软件工程, 2020(9):38-39.

[10] 王春明. 基于 Java 的网上购物系统的设计与实现[J]. 电子技术与软件工程, 2020(20):48-50.

[11] 孙善毅, 朱宁. 基于 SSM 框架的学生信息管理系统的开发与应用[J]. 电子技术与软件工程, 2021(4):43-45.

[12] 裴丽丽. 基于 Python 对豆瓣电影数据爬虫的设计与实现[J]. 电子技术与软件工程, 2019(13):176-177.

[13] 谢志妮. 基于 uni-app 的微信小程序关键技术运用[J]. 电子技术与软件工程, 2021(12):32-33.

[14] Sun C-A, Xiao M-J, Dai H-P, et al. A Reinforcement Learning Based Approach to Partition Testing[J]. Journal of Computer Science and Technology, 2025, 40(1):99-118. DOI:10.1007/s11390-024-2900-7.

[15] Liu X-J, Yu P, Ma X-X. An Empirical Study on Automated Test Generation Tools for Java: Effectiveness and Challenges[J]. Journal of Computer Science and Technology, 2024, 39(3):715-736. DOI:10.1007/s11390-023-1935-5.

致谢

首先，本论文在完成过程中，得到了指导老师在研究思路、论文结构和写作规范方面的悉心指导，在此表示诚挚感谢。然后，感谢父母在学习与生活中给予我的支持和鼓励，使我能够顺利完成本次毕业设计。除此之外,感谢在项目开发和测试过程中给予我帮助的同学与朋友，你们的建议对论文完善起到了积极作用。最后，感谢本文所参考文献的作者，相关研究成果为本课题提供了重要理论依据与技术参考。